ARENA 3.0 Extended Review Packet: DiffusionGemma NVFP4 Proof and BF16 Deferral

Local verification packet

2026-07-01

Verdict

The section 5.5 diffusion-language-model exercises currently have a valid CUDA toy-model learning ladder, deterministic controls, and a strict claim boundary. They now also have a real released-checkpoint DiffusionGemma generation proof for the 24GB route: the pinned NVIDIA NVFP4 checkpoint generated a non-empty answer in an isolated vLLM 0.24.0 runtime on the local RTX 5090 Laptop GPU. The main uv environment remains the requested current stack, torch 2.12.1+cu132 with CUDA 13.2, and vLLM is not installed into it because public vLLM 0.24.0 resolves to torch 2.11.0+cu130. The full Google BF16 DiffusionGemma path remains `deferred` for direct local loading because the required weight shards alone are 51,647,701,024 bytes, or 48.10 GiB, while the machine reports 23.46 GiB of GPU memory.

1 Motivation and Research Question

The motivation is to keep the extension faithful to original ARENA standards: students should implement the core algorithm, test it against explicit references and controls, and only then connect it to a real modern model. The research question is: can this extension honestly claim an ARENA-style DiffusionGemma exercise on a single 24GB laptop GPU?

The current answer is narrower:

- Yes for the pedagogical discrete-diffusion exercises: the implementation has noising, denoising loss, remasking, sampler, activation-trajectory checks, CUDA training, and a shuffled-label negative control.
- Yes for released-checkpoint generation in the intended 24GB route: NVIDIA NVFP4 has a pinned isolated-vLLM proof artifact with prompt, output, runtime, and GPU metadata.
- Deferred for direct Google BF16 local inference on this machine: the model weights alone exceed the local VRAM target.
- Still not claimed: DiffusionGemma denoising-step activation capture, diffusion-time patching, quality evaluation, throughput benchmarking, or full interpretability parity with original ARENA-style mechanistic investigations.

2 Background

DiffusionGemma is a text diffusion language model. Unlike a standard autoregressive transformer that commits to one next token at a time, a discrete diffusion language model starts with masked or

noisy tokens and iteratively denoises a block. The course exercise teaches this by first implementing a small block denoising model on generated integer-token data, then checking whether the released DiffusionGemma artifacts can be used locally.

The local hardware target for this fork is a single 24GB CUDA GPU. The current machine reports:

- GPU: NVIDIA GeForce RTX 5090 Laptop GPU.
- GPU memory reported by torch: 23.46 GiB.
- uv-managed torch stack: torch 2.12.1+cu132, CUDA 13.2, torchvision 0.27.1+cu132.
- ModelOpt: nvidia-modelopt 0.44.0 is installed.
- Isolated vLLM proof runtime: vLLM 0.24.0, torch 2.11.0+cu130, CUDA 13.0.

3 Dataset and Exercise Structure

The accepted part of section 5.5 uses a generated conditional copy-pair grammar. Each example contains two protected prefix tokens and four suffix tokens. The label rule is:

$$[a, b] \mapsto [a, a, b, b].$$

For example, the protected prefix $[3, 5]$ gives the full sequence $[3, 5, 3, 3, 5, 5]$. The verification report records 80 training examples, 20 held-out examples, vocabulary size 11, sequence length 6, mask token id 10, and a deterministic shuffled-label control.

4 Methodology

The toy exercise checks the following mathematical objects:

1. A linear masking schedule p_t that controls how much of the suffix is masked at denoising step t.
2. A masked-token denoising objective:

$$L = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid x_{\setminus M}, t),$$

   where M is the set of masked positions.
3. A confidence-remasking sampler that repeatedly predicts suffix tokens and re-masks low-confidence positions.
4. A commitment-time trace that records when each suffix position becomes stable during denoising.

This tests the teaching mechanism, not the released checkpoint. The released checkpoint path is checked separately by exact repository revisions, required files, shard counts, runtime metadata, and an explicit boolean `diffusiongemma_generation_ready`.

Pseudocode

```
for batch in generated_copy_pair_examples:
    t = sample_diffusion_step()
    masked = apply_linear_suffix_mask(batch.tokens, t)
```

```
    logits = denoiser(masked.tokens, t)
    loss = cross_entropy(logits[masked.positions],
    ↪  batch.tokens[masked.positions])
    update_model(loss)

for heldout_batch in heldout_examples:
    sample = confidence_remasking_sampler(denoiser, heldout_batch.prefix)
    record_exact_match_suffix_accuracy(sample, heldout_batch.target)
    record_commitment_times_and_entropy(sample.trajectory)

run_same_training_path_with_shuffled_suffix_labels()
assert shuffled_label_accuracy_is_low()
assert released_diffusiongemma_generation_ready
assert nvfp4_generation_proof_runtime_is_isolated_from_main_uv
assert bf16_direct_local_loading_is_deferred_for_24gb
```

5 Metrics Primer and Results

The primary toy metrics are held-out masked-token accuracy, sampler exact-match rate, shuffled-label control accuracy, and measured peak VRAM. Held-out accuracy checks whether the denoiser learned the generated copy-pair rule. Sampler exact-match checks whether iterative denoising reconstructs the entire suffix. The shuffled-label control should fail, showing the model is not passing through an accidental shortcut. Matthews correlation coefficient (MCC) is not applicable because this is not a binary classification task.

Metric	Current observed value
Section 5.5 verification accepted	true
Toy CUDA held-out masked accuracy	1.0
Toy CUDA sampler exact match	1.0
Shuffled-label control accuracy	0.0714
Toy CUDA peak VRAM	0.0737 GiB
DiffusionGemma released-checkpoint generation ready	true
NVFP4 isolated vLLM proof	true, vLLM 0.24.0 / torch 2.11.0+cu130
Main uv vLLM generation support	false, main uv stays torch 2.12.1+cu132
Google BF16 weight bytes required	51,647,701,024 bytes, 48.10 GiB
NVIDIA NVFP4 weight bytes required	18,823,855,888 bytes, 17.53 GiB

6 Qualitative Examples

The toy qualitative example is a generated grammar row: prefix $[3, 5]$ implies target suffix $[3, 3, 5, 5]$, so the full sequence is $[3, 5, 3, 3, 5, 5]$. Passing toy samples should reconstruct this suffix from masked states, while the shuffled-label control should not learn a stable rule.

The released-checkpoint proof used this prompt:

> In one concise sentence, explain why mechanistic interpretability experiments need negative controls.

The pinned NVFP4 checkpoint generated:

> Negative controls are necessary to ensure that observed model behaviors are causally linked to the specific mechanisms being tested rather than artifacts, chance correlations, or noise.

This is a one-prompt runtime proof, not an output-quality benchmark.

7 Artifact Status

Item	Status	Evidence and implication
Google BF16 DiffusionGemma	`deferred`	`google/diffusiongemma-26B-A4B-it` at revision `0f28bc42f588fbd8f71e08102b1c3960298a1358`. The report requires 11 safetensor shards totaling 48.10 GiB. Local shards are incomplete. This is too large for direct local loading on a 23.46 GiB GPU, before activations and runtime overhead.
NVIDIA NVFP4 DiffusionGemma	`proven in isolated runtime`	`nvidia/diffusiongemma-26B-A4B-it-NVFP4` at revision `2ea837236295d617ac27f8c17d61228081932c40`. The two weight shards total 17.53 GiB and are the intended 24GB path. The local shards are complete, the model path matches the pinned revision, and `artifacts/diffusiongemma_vllm_probe.json` records a real generation on the 23.46 GiB RTX 5090 Laptop GPU.
Transformers NVFP4 loading	`not the active proof path`	The checkpoint advertises `quant_method: modelopt`. The current Transformers quantization registry rejects this with "Unknown quantization type, got modelopt". Config, tokenizer, processor, and model-class support remain useful preflight evidence, but the generation proof currently comes from vLLM, not Transformers direct loading.
vLLM dependency path	`isolated`	`uv pip install -dry-run vllm -prerelease allow` resolves vLLM 0.24.0, but would replace torch 2.12.1+cu132 with torch 2.11.0 and replace CUDA 13.2 packages with CUDA 13.0 packages. That conflicts with the requested latest cu132 torch stack, so vLLM was installed in `/tmp/arena-diffusiongemma-vllm` for the proof run instead of in the main uv environment.

8 Supported and Unsupported Claims

Supported now. The course exercise has a working CUDA-trained tiny discrete diffusion language model, held-out testing, a sampler reconstruction metric, activation-trajectory checks, and a negative control that fails as expected. It also has real NVIDIA NVFP4 DiffusionGemma generation evidence in an isolated vLLM runtime under the 24GB GPU target.

Unsupported now. The course must not claim BF16 direct local inference, DiffusionGemma denoising-step activation capture, diffusion-time patching, throughput, broad output quality, or full interpretability parity from this one-prompt proof. It also must not treat mock generations, fake outputs, config-only checks, or placeholder artifacts as substitutes for artifact-backed runs.

9 Implementation Pointers

- `chapter5_modern_architectures/exercises/part5_diffusion_language_models/verification_report.json` contains the accepted toy CUDA metrics, BF16 deferral, and NVFP4 isolated-vLLM generation proof fields.
- `chapter5_modern_architectures/exercises/part5_diffusion_language_models/artifacts/diffusiongemma_vllm_probe.json` is the compact proof artifact for the real NVFP4 generation run.
- `arena_ext/diffusion_lm.py` computes the DiffusionGemma readiness report and exposes the generation-ready boolean.
- `arena_ext/gated_artifacts.py` pins the exact Google BF16 and NVIDIA NVFP4 repository revisions and required files.
- `scripts/prepare_diffusiongemma_artifacts.py` inspects or downloads the pinned artifacts. The conservative command is:

```
BNB_CUDA_VERSION=130 uv run python scripts/prepare_diffusiongemma_artifacts.py
↪  \
  --artifact nvfp4 --download --require-local --max-workers 1
```

- `scripts/run_diffusiongemma_vllm_probe.py` runs the real generation probe from an isolated vLLM Python executable.
- `FOR_REVIEW/tables/diffusiongemma_readiness.csv` is the compact evidence table for this packet.

10 Validation Commands

```
uv run python - <<'PY'
import torch, torchvision, modelopt
print(torch.__version__, torch.version.cuda)
print(torch.cuda.get_device_name(0))
props = torch.cuda.get_device_properties(0)
print(round(props.total_memory / 1024**3, 2))
print(torchvision.__version__)
print(modelopt.__version__)
PY

BNB_CUDA_VERSION=130 uv run python -m pytest -q \
  tests/test_diffusion_lm.py \
  chapter5_modern_architectures/exercises/part5_diffusion_language_models/tests.
  ↪  py
```

```
BNB_CUDA_VERSION=130 uv run python scripts/run_extension_verification_reports.py
↪  \
  --section 1.6 --section 5.1 --section 5.5 --section 5.6 --max-vram-gb 24

BNB_CUDA_VERSION=130 uv run python scripts/prepare_diffusiongemma_artifacts.py \
  --artifact all --json

/tmp/arena-diffusiongemma-vllm/bin/python
↪  scripts/run_diffusiongemma_vllm_probe.py \
  --model /home/bitwise/.cache/huggingface/hub/models--nvidia--diffusiongemma-26
  ↪  B-A4B-it-NVFP4/snapshots/2ea837236295d617ac27f8c17d61228081932c40
  ↪  \
  --prompt 'In one concise sentence, explain why mechanistic interpretability
  ↪  experiments need negative controls.' \
  --max-tokens 128 --max-model-len 4096 --max-num-seqs 1 \
  --gpu-memory-utilization 0.85 --canvas-length 256 --use-chat-template \
  --output-json chapter5_modern_architectures/exercises/part5_diffusion_language
  ↪  _models/artifacts/diffusiongemma_vllm_probe.json

uv pip check
uv pip install --dry-run vllm --prerelease allow
tectonic FOR_REVIEW/FOR_REVIEW.tex --outdir FOR_REVIEW
pdftotext FOR_REVIEW/FOR_REVIEW.pdf -
CHECK=/home/bitwise/.codex/skills/for-review-pdf/scripts/check_for_review_pdf.py
python "$CHECK" FOR_REVIEW/FOR_REVIEW.pdf --tex FOR_REVIEW/FOR_REVIEW.tex
```

11 Limitations and Blockers

- The BF16 model is intentionally deferred for local 24GB execution. That deferral should be reviewed, but it does not block the intended 24GB NVFP4 route.
- The installed Transformers stack still cannot directly consume the NVIDIA ModelOpt NVFP4 quantization config; vLLM is the proven local runtime.
- vLLM currently resolves to a stack change that downgrades torch and CUDA packages, so it was not installed into the main uv environment. The proof uses an isolated environment instead.
- The NVFP4 proof is one deterministic prompt. It proves loading and generation on this GPU, not output quality, throughput, or a prompt suite.
- The exercise still needs released-checkpoint denoising-step output/activation capture and a matched random-control diffusion-time patching exercise before claiming mechanism-level parity.

12 Questions for Expert Review

1. Is the **`deferred`** marker for direct BF16 local inference accepted as a reasonable 24GB hardware deferral?

2. Is the isolated-vLLM split acceptable for the 24GB local route while the main uv environment remains torch 2.12.1+cu132?
3. Should the next sprint prioritize a ModelOpt-aware Transformers path, a documented isolated-vLLM workflow, or a containerized runtime?
4. What minimum evidence should be required after this one-prompt proof: a prompt suite, throughput/VRAM measurements, denoising trajectories, or a patching intervention?
5. What output-quality and reproducibility checks would make the notebook feel closest to original ARENA quality rather than a thin model-loading demo?

A Artifact Index

- `Extension-Roadmap.md`: roadmap requirement that quantized DiffusionGemma run under the 24GB budget and that every notebook end with a verification block.
- `docs/local_gpu_setup.md`: local setup notes and DiffusionGemma artifact preparation command.
- `docs/method_registry.md`: method registry row marking DiffusionGemma local inference as NVFP4-proven in an isolated vLLM runtime.
- `chapter5_modern_architectures/exercises/part5_diffusion_language_models/artifacts.lock.yml`: pinned claim scope, controls, revisions, and expected metrics.
- `chapter5_modern_architectures/exercises/part5_diffusion_language_models/expected_outputs/reference_metrics.json`: expected metric thresholds and current generation-ready flag.
- `scripts/run_diffusiongemma_vllm_probe.py`: exact probe runner for the isolated vLLM generation artifact.
- `chapter5_modern_architectures/exercises/part5_diffusion_language_models/artifacts/diffusiongemma_vllm_probe.json`: prompt, output, vLLM version, torch/CUDA version, GPU name, memory, and timing for the real NVFP4 proof.